Conditioning generation on PCA coordinates: post-mortem and candidate methods

AttentionDCA / J-domains — proposal for the next experiment

October 1, 2026

Abstract

An earlier study appended the discretised PCA coordinates to each sequence as two extra sites and learned a joint coupling tensor over sequence and label, steering generation with a separate inverse temperature β_{PCA}. It works only partially: steering requires $\beta_{\mathrm{PCA}} \gtrsim 10$, and beyond that the samples stop being J-domains. This note diagnoses *why* that happens, then sets out four ways to do conditional generation properly. The central observation is that a PCA coordinate is an *exactly linear* function of the one-hot sequence, so conditioning on it needs no learning at all: the constraint can be written in closed form as a Potts term and added to the existing model. Nothing here is implemented yet; a common evaluation protocol is given at the end. A secondary section covers the separate question of the unconditional model's mode weights.

1 Two goals that are often confused

(G1) Conditional generation. "Give me sequences at this place in PCA space." Success means the samples land near the target *and* remain valid J-domains. This is the goal of the earlier study.

(G2) Correct mode weights. "When I sample freely, each sub-family should appear with its natural frequency." Our current model fails this for one cluster: C0 carries 14.1% of the natural weight but receives 5.1% of the samples ($\Delta F = +1.01$ nats).

They are different problems. G1 is solved by adding a constraint at sampling time; G2 is a property of the learned energy function and is not fixed by conditioning — though, usefully, a good solution to G1 gives a *measurement* tool for G2 (§4). Sections 2–3 address G1; §5 summarises G2.

2 Post-mortem: why appending discretised coordinates underperforms

In the earlier work the 35×35 discretised PCA grid was appended as two extra sites with an enlarged alphabet ($\bar{q} = 35$), giving a joint tensor J of shape $(\bar{q}, \bar{q}, L{+}2, L{+}2)$, and sampling used $P(a_i \mid \boldsymbol{a}_{-i}, \boldsymbol{z}) \propto \exp\big(\sum_{j \le L} J_{ij}(a_i, a_j) + \beta_{\mathrm{PCA}} \sum_{j > L} J_{ij}(a_i, z_j)\big)$. Four structural problems explain the reported behaviour, and none of them is a tuning issue.

1. **The label is outvoted 2 to L.** In the conditional for site i the label contributes 2 of $L + 2 = 65$ coupling terms, about 3% of the local field. That is exactly why steering only begins around $\beta_{\mathrm{PCA}} \approx 10$. But β_{PCA} multiplies *learned* couplings, whose directions in sequence space are arbitrary wherever the data were thin; amplifying them by 10–50 pushes sequences along those arbitrary directions, which is the observed "shifts too far right, loses the HPD motif". There is no β_{PCA} that both steers and preserves the protein, because the two effects are driven by the same amplified parameters.

2. **One-hot bins throw away the geometry of PCA space.** Bins 20 and 21 are adjacent in PC_1 but are unrelated symbols in a Potts alphabet: nothing ties their columns together. With $35 \times 35 = 1225$ cells and $\sim 10^4$ training sequences, most cells hold a handful of sequences or none, so the label couplings are estimated from almost no data and cannot interpolate. This is the direct explanation of the coordinate-generation figures: conditioning on a true training sequence and sampling its coordinates puts mass on one or two spurious cells (Model 1/2) or is nearly uniform (Model 3), instead of recovering the true cell.

3. **The label is a deterministic function of the sequence.** $\boldsymbol{z} = f(\boldsymbol{x})$ exactly, so the true $p(\boldsymbol{z} \mid \boldsymbol{x})$ is a delta. A handful of pairwise terms to a 35-state variable cannot represent a delta, so the fit is forced to be wrong; the model is being asked to learn something it cannot express, and the coordinate-generation test measures precisely that failure.

4. **It is unnecessary.** The quantity being conditioned on is already an exact linear function of the model's own variables (§3), so learning it from data only introduces estimation error where a closed form exists.

3 The key fact: PC is linear in the one-hot sequence

Let $\boldsymbol{x} \in \{0,1\}^{Lq}$ be the one-hot encoding, $x_{ia} = 1$ iff site i carries letter a. The PCA used throughout this project is fitted on exactly these vectors, so for component c

$$\mathrm{PC}_c(\boldsymbol{x}) = \sum_{i,a} V_{c,ia}\, x_{ia} - m_c, \qquad m_c = \sum_{i,a} V_{c,ia}\, \mu_{ia}, \tag{1}$$

with V the loadings and μ the training mean. This is not an approximation: evaluating (1) against `sklearn`'s `transform` on our test set reproduces it to 0.0 (machine zero).

Therefore a harmonic restraint towards a target $\boldsymbol{t}$ in PCA space,

$$E_\lambda(\boldsymbol{x}) = E(\boldsymbol{x}) + \frac{\lambda}{2} \sum_c \big(\mathrm{PC}_c(\boldsymbol{x}) - t_c\big)^2, \tag{2}$$

is *exactly a Potts energy*: expanding the square gives a pairwise part and a field part,

$$\Delta J_{ia,jb} = -\lambda \sum_c V_{c,ia} V_{c,jb}, \qquad \Delta h_{ia} = \lambda \sum_c (t_c + m_c)\, V_{c,ia}, \tag{3}$$

i.e. a rank-C correction to the coupling matrix plus a field. No training, no new parameters, no β_{PCA} to tune blindly — λ is a restraint stiffness with a clear meaning, as in umbrella sampling.

Why this behaves better than the learned version. Increasing λ constrains the sequence only along the C directions V_c; every direction orthogonal to them stays governed by the protein energy $E(\boldsymbol{x})$. The failure mode of the earlier approach — amplifying arbitrary learned directions until the protein breaks — cannot occur, because the constrained direction is the PCA direction itself, by construction. Quality still degrades as $\lambda \to \infty$ (the constraint eventually dominates), but gracefully and for an understandable reason.

Scale, for intuition. On our clean split, PC_1 spans 6.9 across the family with standard deviation 2.06, while a single substitution moves it by 0.125 on average (at most 0.42). Reaching a distant target therefore requires tens of coordinated substitutions, which is why a weak restraint does nothing and why the restraint must be applied *during* sampling rather than as a post-hoc filter.

C1 — Analytic harmonic restraint (recommended)

Implement (2) directly in the existing sampler. Each chain keeps a running PC_c; changing site i from a to a' shifts it by $V_{c,ia'} - V_{c,ia}$, so the conditional logits cost $O(q\,C)$ extra per site update — negligible.
Cost: ∼30 lines in `sampler.py`, no retraining, minutes to run. Expected: samples concentrated near $\boldsymbol{t}$ with a controlled trade-off; a monotone, interpretable λ curve replacing the non-monotone β_{PCA} behaviour. Risks: a harmonic restraint targets a *point*; for a region, restrain to the region's centre with a soft λ, or use a flat-bottomed restraint (zero penalty inside the target cell, quadratic outside). Also, PC is a 2-D summary explaining only $11.3\% + 4.3\%$ of the variance, so "hit the target" never means "is the right protein" — sequence-quality metrics must be reported alongside.

C2 — Exact conditional by rejection (ground truth)

Sample the unconditioned model, keep only the sequences whose coordinates fall in the target cell. This yields *exactly* $p(\boldsymbol{x} \mid \boldsymbol{z} \in \text{cell})$ with no approximation.
Cost: 20 lines, no retraining. Expected: the reference answer every other method should be compared against — for the first time we would know what the conditional distribution actually is. Risks: efficiency falls with the natural density of the target; for a cell in the dense part of the family it is cheap, for the far tail it is hopeless. This is the method's honest limitation, and it is exactly why C1/C3 exist — but it makes C2 the right validation for *reachable* targets, including C0.

C3 — Learned conditional field (if a learned model is wanted)

Keep the coordinates *continuous* and let them bias every site through a field rather than through two coupling columns:

$$p(x_i = a \mid \boldsymbol{x}_{-i}, \boldsymbol{z}) \propto \exp\Big(\sum_{j \neq i} J_{ij}(a, x_j) + \boldsymbol{w}_{ia} \cdot \phi(\boldsymbol{z})\Big), \qquad \phi(\boldsymbol{z}) = (1, z_1, z_2, z_1^2, z_2^2, z_1 z_2), \tag{4}$$

trained by conditional pseudo-likelihood on $(\boldsymbol{x}, \boldsymbol{z} = f(\boldsymbol{x}))$ pairs. Parameters: $|\phi|\, Lq \approx 8\,000$, a 4% increase. This repairs problems 1 and 2 of §2: the label now reaches *every* site, and smoothness in $\boldsymbol{z}$ is built into the parametrisation so neighbouring coordinates share statistical strength.
Cost: moderate — conditional field in model, trainer and sampler; 1 training run (≈ 40 min). Expected: steering without a large temperature, and a model that can also answer $p(\boldsymbol{z} \mid \boldsymbol{x})$ sensibly. Risks: still learns what C1 knows exactly, so it is only worth it if the aim is a *learned* conditional model (e.g. to later condition on something that is *not* a linear function of the sequence — phenotype, host, temperature). That is the real argument for C3, and it is a good one.

C4 — Repair the label-as-sites model (if continuity with the old work matters)

Two targeted fixes: (i) give the label an ordinal/smooth encoding — e.g. replace the 35-state one-hot bin by a small set of basis features, or tie neighbouring bins with a smoothness penalty $\sum_z \|J(\cdot, z{+}1) - J(\cdot, z)\|^2$; (ii) rebalance the 2-vs-L asymmetry by replicating the label over R sites or by scaling the label block at *training* time rather than inflating β_{PCA} at sampling time.
Cost: 1 training run plus the smoothness term. Expected: coordinate generation that recovers the true cell, which is the clean diagnostic the old report found broken. Risks: keeps the discretisation that caused the trouble; strictly dominated by C1 for this particular conditioning variable.

	learning needed	effort	β_{PCA} pathology	what it gives
C1 harmonic restraint	none (closed form)	low	no	controlled steering, free-energy maps
C2 rejection	none	trivial	no	the exact conditional, as ground truth
C3 conditional field	yes	medium	no	a learned conditional, extensible to other labels
C4 repaired labels	yes	medium	reduced	continuity with the previous study

Table 1: Conditional-generation options. C1 and C2 require no retraining.

4 A bonus: the restraint measures the model's error everywhere

Running C1 at a grid of targets with a moderate λ is umbrella sampling along $(\mathrm{PC}_1, \mathrm{PC}_2)$. Reweighting the windows (WHAM, or simply the known biasing potential) gives the model's free-energy surface $F_{\mathrm{model}}(\boldsymbol{z})$, which can be compared directly with the natural one, $-\ln \rho_{\mathrm{nat}}(\boldsymbol{z})$ estimated from the reweighted training sequences. The difference is a *map* of where the model misplaces probability — a much richer object than the three per-cluster numbers we currently report, and it would show whether the C0 deficit is a uniform offset of that basin or a distortion of its shape. Cost: one C1 implementation plus a grid of runs.

5 Separately: the unconditional mode weights (G2)

For completeness, the options for the unconditional problem, unchanged from the previous note: **M1** per-cluster reweighting of the training set by $e^{\gamma \Delta F_k}$ (a control loop; cheap, but C0 has only $M_{\mathrm{eff}} = 464$ against 217 728 parameters, so it risks over-fitting); **M2** add explicit fields h_i (cheap, a correctness fix in its own right); **M3** an explicit mixture with a hand-set prior π_k (correct by construction, but imposes rather than learns the prior); **M4** likelihood-based fine-tuning by persistent contrastive divergence, whose gradient $\langle\cdot\rangle_{\mathrm{data}} - \langle\cdot\rangle_{\mathrm{model}}$ contains exactly the term pseudo-likelihood omits (the principled fix, highest effort); **M5** resampling generated pools to the natural cluster shares (a control, not a fix).

Note that C1/C2 give a cheap partial answer to G2 as well: generating a pool whose cluster proportions are forced to the natural ones isolates how much of the full-set statistics gap is due to mode weight alone. Since restricting to the covered clusters already lifts $r(C_{ij})$ from 0.871 to 0.918, we can predict the outcome and check it.

6 Evaluation protocol (fixed in advance)

For conditional generation, every method is reported on the same two axes, because the previous study's difficulty was precisely that improving one destroys the other:

- **On-target accuracy:** mean and spread of $\|\mathrm{PC}(\boldsymbol{x}) - \boldsymbol{t}\|$ over the generated pool; fraction landing in the target cell.

- **Sequence quality, at matched accuracy:** $r(f_i)$ and $r(C_{ij})$ against the *natural sequences in the same region* (not the whole family — the earlier report noted this distinction and it is the right one); retention of the HPD motif; mean distance to the nearest natural sequence; held-out energy of the samples.
- **The trade-off curve itself:** sweep λ (or β_{PCA}) and plot quality against accuracy. A method is better if its curve dominates, which is a fairer comparison than any single operating point.
- **Against ground truth:** wherever C2 is affordable, report the gap between each method's conditional and the exact one.
- **Targets:** at least one dense target and one in C0, since the sparse region is where methods differ.

Recommendation. C1 + C2 together: C2 fixes what the right answer is, C1 provides a cheap and well-behaved way to reach it, and neither needs retraining, so both can be done and evaluated quickly. Add **C3** if the goal is a learned conditional model that will later be conditioned on labels which are *not* functions of the sequence — that is the case where the analytic route is unavailable and the extra machinery earns its keep.